

Explicación de main.py

TP Simulación de Colas — Grupo 22
Sistema de Registro Dactilar — Municipalidad de Río Cuarto

1. Visión general de main.py

main.py es el archivo de interfaz gráfica de la aplicación. Su responsabilidad es mostrar los resultados que produce **simulacion.py** en una ventana de escritorio construida con **PyQt5**.

El archivo se divide en seis bloques principales:

1. Constantes, definición de columnas y tema visual.
2. *SimModel* — el modelo de datos virtual que alimenta la tabla.
3. *GroupedHeader* — el encabezado de dos niveles dibujado a mano.
4. *SimWorker* — el hilo aparte que corre la simulación sin congelar la UI.
5. *MainWindow* — la ventana principal: carga el layout, conecta señales, filtra filas y muestra estadísticas.
6. Helpers y métodos auxiliares.

No hay servidor web, no hay base de datos, no hay navegador. Todo corre en un único proceso de escritorio.

2. Constantes y estructura de columnas

2.1 La lista COLS

COLS es la lista que define todas las columnas estáticas de la tabla, en orden. Cada elemento es una tupla (**clave**, **título**): la clave es el nombre interno que usa el modelo para saber qué dato leer de cada fila, y el título es lo que se muestra en el encabezado.

```
COLS = [  
    ("num",      "#"),          ("reloj",    "Reloj"),          ("evento",   "Evento"),  
    ("rnd_le",   "RND"),        ("t_le",     "T. Entrada"),    ("prox_emp", "Prox. Evento"),  
    ("rnd_lt",   "RND"),        ("t_lt",     "T. Entrada"),    ("prox_tec", "Prox. Evento"),  
    ("rnd_at",   "RND"),        ("t_at",     "T. Atencion"),  
    ("fin_at_1", "Fin At. 1"),   ("fin_at_2", "Fin At. 2"),  
    ("fin_at_3", "Fin At. 3"),   ("fin_at_4", "Fin At. 4"),  
    ("rnd_mt",   "RND"),        ("t_mt",     "T. Mantenim."),  ("fin_mant", "Fin Mantenim."),  
    ("t1_est",   "Estado"),      ("t1_pend",  "Pendiente"),  
    ... # terminales 2, 3, 4  
    ("cola",     "Cola"),  
    ("tec_est",  "Estado"),      ("tec_term", "Terminal"),  
    ("n_lleg",   "Llegadas"),    ("n_rt",     "Se van"),        ("pct_se_van", "% se van"),
```

```

    ("acum_esp", "Acum. espera"), ("n_esp", "Cnt. espera"), ("prom_esp", "Prom. espera"),
]

```

Esta lista es la única fuente de verdad sobre qué columnas existen y en qué orden. Las columnas de empleados **no** están acá porque son dinámicas: se calculan en cada actualización de la tabla según qué empleados aparecen en las filas visibles.

Claves como 'fin_at_1' o 't2_est' son interpretadas por `_spec` para extraer el dato correcto del índice de la lista interna de terminales.

2.2 BASE_GROUPS — grupos del encabezado

Define los grupos de columnas para el encabezado de dos niveles: nombre del grupo, color de fondo, color de texto y cantidad de columnas que abarca. Por ejemplo, 'Terminal 1' abarca 2 columnas (Estado y Pendiente) con fondo cyan.

2.3 DARK — el tema visual

El tema visual de toda la aplicación se define en la constante **DARK**, una cadena de texto con estilos CSS que PyQt5 llama *stylesheet*. Se aplica una sola vez al iniciar la app:

```
app.setStyleSheet(DARK)
```

El tema usado es **Catppuccin Mocha**, una paleta oscura. Algunos de los colores principales:

- Fondo de ventana y widgets: **#1e1e2e** (azul muy oscuro).
- Fondo de tablas: **#181825** (aún más oscuro).
- Texto principal: **#cdd6f4** (blanco lavanda).
- Botón principal: **#cba6f7** (violeta).
- Bordes y separadores: **#45475a** (gris medio).

El stylesheet se aplica globalmente a toda la aplicación, por eso todos los widgets heredan el mismo tema sin necesidad de configurarlos uno por uno.

2.4 Diccionarios de colores por estado

Además del tema general, hay tres diccionarios que mapean estados a colores de texto para las celdas de la tabla:

```

TERM_COLOR = {"Libre": "#89dceb", "Ocupada": "#fab387", "Siendo mantenida": "#f38ba8"}
TEC_COLOR  = {"Descansando": "#9399b2",
              "Esperando Terminal Libre": "#f9e2af",
              "Realizando Mantenimiento": "#f38ba8"}
EMP_COLOR  = {"SA": "#a6e3a1", "EA": "#f9e2af", "x": "#f38ba8"}

```

Cyan para libre, naranja para ocupada, rojo para en mantenimiento. Verde para siendo atendido, amarillo para esperando, rojo para el empleado que se va.

3. Conexión con main.ui

main.ui es un archivo XML generado con Qt Designer. Define la estructura visual de la ventana: qué widgets hay, cómo están ubicados y cómo se llaman. No contiene lógica de ningún tipo.

main.py lo carga con una sola línea:

```
uic.loadUi(os.path.join(os.path.dirname(__file__), 'main.ui'), self)
```

Esto crea automáticamente todos los widgets definidos en el `.ui` como atributos del objeto *self*. Si en el `.ui` hay un botón llamado **btn_simular**, después de cargar el archivo ese botón es accesible directamente como **self.btn_simular**.

Los widgets principales accedidos desde el código son:

- **txt_media_llegada**, **txt_maxCola**, **txt_atn_min/max**, **txt_mant_min/max**, **txt_tec_min/max**: parámetros del sistema.
- **txt_tiempo_max**: duración de la simulación en minutos.
- **btn_simular**: arranca la simulación.
- **btn_copiar**: copia los datos al portapapeles.
- **prg_sim**: barra de progreso indeterminada durante la simulación.
- **lbl_info**: muestra mensajes de estado, errores o resultados.
- **tbl_vector**: tabla principal con todas las filas.
- **tbl_ultima**: tabla de una sola fila (estado final).
- **txt_hora_desde**, **txt_cant_filas**: filtros de visualización *j* e *i*.
- **pnl_stats**: panel donde se crean las tarjetas de estadísticas en Python.

4. Conexión con `simulacion.py`

main.py importa **simulacion.py** como un módulo normal:

```
import simulacion as sim
```

4.1 Configurar parámetros

Antes de simular, lee los campos de texto del usuario y sobrescribe las variables globales de **simulacion.py** directamente:

```
sim.MEDIA_LLEGADA_EMP = p['media']
sim.MAX_COLA           = p['maxCola']
sim.ATN_MIN, sim.ATN_MAX = p['atn_min'], p['atn_max']
sim.MANT_MIN, sim.MANT_MAX = p['mant_min'], p['mant_max']
sim.TEC_MIN,  sim.TEC_MAX  = p['tec_min'], p['tec_max']
```

4.2 Limpiar y simular

Llama a **resetear_estado()** para partir desde cero, y luego a **simular(tiempo_max)** que corre el loop de eventos y llena **sim.vector_estado** con todas las filas.

```
sim.resetear_estado()
sim.simular(p['tiempo_max'])
```

4.3 Leer los resultados

Al terminar, lee **sim.vector_estado** (lista de dicts, uno por fila), **sim.reloj** (tiempo final) y **sim.iteracion** (eventos procesados):

```
self.todas_filas = sim.vector_estado
self._relojes = [f['reloj'] for f in self.todas_filas]
```

A partir de ese momento **main.py** trabaja solo con esos datos. No vuelve a llamar a **simulacion.py** hasta que el usuario presione *Simular* de nuevo.

5. SimModel — el modelo virtual de datos

PyQt5 separa los datos de la vista. La tabla (*QTableView*) no guarda datos; le pide al **modelo** qué mostrar en cada celda cuando lo necesita. Eso se llama **renderizado virtual**: aunque haya 100.000 filas, solo se renderizan las celdas visibles en pantalla en cada momento.

SimModel hereda de *QAbstractTableModel* e implementa el contrato mínimo que PyQt5 requiere:

```
def rowCount(self, parent=None): return len(self.rows)
def columnCount(self, parent=None): return len(self._specs)
def headerData(...): # devuelve el titulo de cada columna
def data(index, role): # devuelve el contenido y el color de cada celda
```

5.1 set_content — cargar datos

Cuando llegan datos nuevos, se llama a **set_content(rows, cols)**. Recibe la lista de filas y la lista de columnas (clave + título). Construye **self._specs**: una especificación por columna que describe cómo leer su valor.

5.2 _spec — cómo interpretar cada columna

Analiza la clave de cada columna y devuelve una tupla que describe cómo extraer el valor. Los tipos posibles son:

- **scalar**: columnas simples como *reloj*, *cola*, *tec_est*. Se lee directamente del dict de la fila con la clave dada.
- **termlist**: columnas de terminales como *t1_est*, *fin_at_2*. Se lee de una lista interna de la fila usando el índice (número de terminal - 1).
- **pct**: calcula el porcentaje de rechazados: $n_{rt} / n_{lleg} * 100$.
- **prom**: calcula el promedio de espera: $acum_esp / n_esp$.
- **emp**: columnas dinámicas de empleados. Extrae estado, hora de inicio de espera y terminal del empleado con ese ID.

pct y prom se calculan en el modelo, no en simulacion.py. simulacion.py solo guarda los valores crudos y el formateo queda del lado de la GUI.

5.3 _text — extraer y convertir el valor de una celda

Es el método que realmente obtiene el valor a mostrar usando la spec. El caso más importante es el de los empleados (**emp**):

```
if kind == 'emp':
    tup = f['emp'].get(a)          # busca al empleado por su ID
    if tup is None: return ''      # el empleado no existe en esta fila
    estado, hll, tid = tup
```

```
fue = estado == 'AT'          # 'fue' = ya termino, se va
if b == 'estado': return 'x' if fue else estado
if b == 'hora':   return '-' if fue else self._fmt(hll)
return '-' if fue else self._fmt(tid)
```

Cuando el estado del empleado es **'AT'** (termino, se va), la columna Estado muestra **'x'** en lugar del estado interno, y las columnas de hora y terminal muestran **'-'** porque ya no tiene sentido mostrar esos datos. La **'x'** es lo que se ve en rojo en la tabla.

5.4 _fmt — formateo final de valores

```
if v is None:          return '-'
if v is True:          return 'SI'
if v is False:         return 'NO'
if isinstance(v, float): return trunc2_str(v)
return str(v)
```

Los floats se truncan (no redondean) a 2 decimales con **trunc2_str**, que usa *math.floor* para garantizar truncado verdadero, tal como pide el enunciado. Los booleanos se muestran como SI/NO. Los None como guion.

5.5 Colores de celdas

Cuando PyQt5 pide el color de una celda (rol *ForegroundRole*), el modelo obtiene el texto de esa celda y lo busca en el diccionario de colores correspondiente a su columna (TERM_COLOR, TEC_COLOR o EMP_COLOR). Si encuentra una entrada, devuelve ese color; si no, la celda queda con el color de texto por defecto.

6. GroupedHeader — encabezado de dos niveles

El encabezado estándar de PyQt5 muestra una sola fila de títulos. Esta aplicación necesita dos filas: la superior con el nombre del grupo (coloreado) y la inferior con el título de cada columna individual.

GroupedHeader hereda de *QHeaderView* y sobreescribe **paintEvent**: dibuja a mano cada celda con un *QPainter* en lugar de usar el pintado estándar.

6.1 set_groups — definir la estructura

Recibe una lista de tuplas (nombre, color_fondo, color_texto, cantidad_columnas) y las convierte en rangos (columna inicio, columna fin). El paintEvent usa esos rangos para saber qué segmento del encabezado pintar de qué color.

6.2 paintEvent — el dibujado

Por cada grupo visible en pantalla, dibuja dos rectángulos:

- **Fila inferior** (mitad de abajo): fondo gris oscuro, texto con el título individual de cada columna.
- **Fila superior** (mitad de arriba): fondo del color del grupo, texto con el nombre del grupo centrado y en negrita.

Solo dibuja lo que está visible en el viewport, optimizando el rendimiento en tablas con muchas columnas.

6.3 Columnas dinamicas de empleados

Las columnas de empleados se agregan al encabezado dinámicamente cada vez que se actualiza la tabla. **update_table** calcula qué empleados aparecen en las filas visibles y construye sus grupos:

```
emp_groups = [(f'Empleado {c}', '#c6a6f7', '#1e1e2e', 3) for c in emp_ids]
```

Cada empleado ocupa 3 columnas: Estado, Hora inicio espera y Terminal. Si en las filas visibles aparecen los empleados 3, 5 y 7, el encabezado mostrará tres grupos: Empleado 3, Empleado 5, Empleado 7.

7. SimWorker — simulacion en hilo aparte

Las interfaces gráficas en PyQt5 tienen un **hilo principal** que es el único que puede actualizar la UI. Si se corre una operación pesada (como 100.000 eventos) en ese hilo, la ventana se congela y no responde.

La simulación se corre en un **hilo separado** con *QThread*. **SimWorker** contiene la lógica a ejecutar en ese hilo:

```
class SimWorker(QObject):
    finished = pyqtSignal()

    def run(self):
        sim.MEDIA_LLEGADA_EMP = p['media']
        ... # configura el resto de parametros
        sim.resetear_estado()
        sim.simular(p['tiempo_max'])
        self.finished.emit() # avisa que termino
```

La señal **finished** es el mecanismo de comunicación segura entre hilos en Qt. Cuando la simulación termina en el hilo secundario, emite esa señal, y el hilo principal la recibe para actualizar la UI sin condiciones de carrera.

7.1 Ciclo de vida del hilo

1. El usuario presiona *Simular*.
2. Se crean un *QThread* y un *SimWorker*.
3. El worker se mueve al hilo nuevo con *moveToThread*.
4. Al iniciarse el hilo, llama a *worker.run()*.
5. Al terminar, emite *finished*, que dispara *_on_sim_done* y luego *thread.quit()*.
6. Cuando el hilo para, *_on_thread_done* hace la limpieza:

```
def _on_thread_done(self):
    if self._worker: self._worker.deleteLater()
    if self._thread: self._thread.deleteLater()
    self._worker = self._thread = None
```

deleteLater es el mecanismo de Qt para liberar objetos de forma segura cuando el hilo ya terminó. Poner ambos en None permite que Python los garbage-collecte y evita referencias colgadas.

Si el usuario presiona Simular mientras ya hay una simulación corriendo, `on_simular` detecta que el hilo está activo y no hace nada: `if self._thread and self._thread.isRunning(): return`

8. `on_simular` — validación y arranque

Es el método que se ejecuta cuando el usuario presiona el botón *Simular*. Tiene dos responsabilidades: validar los parámetros ingresados y, si todo está bien, lanzar la simulación en un hilo aparte.

8.1 Lectura de parámetros con helpers

Los valores de los campos de texto se leen con dos helpers:

```
def _valf(self, w, d):
    try:    return float(w.text().strip()) if w.text().strip() else d
    except: return d

def _vali(self, w, d):
    try:    return int(float(w.text().strip())) if w.text().strip() else d
    except: return d
```

`_valf` lee un float (con valor por defecto *d* si el campo está vacío o tiene texto inválido). `_vali` hace lo mismo pero convierte a entero. Ambos nunca tiran excepción hacia afuera: si el texto no es un número válido, devuelven el valor por defecto silenciosamente.

8.2 Validación de parámetros

Antes de simular, se validan las reglas de negocio y se acumulan los errores:

```
errores = []
if params['media'] <= 0:
    errores.append('La media de llegadas debe ser mayor a 0')
if params['atn_max'] <= params['atn_min']:
    errores.append('Atencion: el max debe ser mayor al min')
... # otras validaciones

if errores:
    self.lbl_info.setStyleSheet('color:#f38ba8;') # rojo
    self.lbl_info.setText(' ' + ' '.join(errores))
    return
```

Si hay errores, se muestran todos juntos en **lbl_info** con texto rojo y la función termina sin simular. Si todo está bien, el label vuelve a verde y se lanza el hilo.

9. Panel de columnas congeladas

La tabla tiene muchas columnas y requiere scroll horizontal. Cuando el usuario hace scroll hacia la derecha, pierde de vista las columnas clave: #, Reloj y Evento.

La solución es usar **dos tablas** que comparten el mismo modelo:

- **_frozen**: tabla secundaria a la izquierda, solo muestra las 3 primeras columnas, sin scroll horizontal.
- **tbl_vector**: tabla principal a la derecha, oculta esas 3 primeras columnas y tiene scroll horizontal completo.

Ambas comparten modelo y modelo de selección, por lo que seleccionar una fila en una la selecciona en la otra. El scroll vertical está sincronizado en ambos sentidos:

```
self.tbl_vector.verticalScrollBar().valueChanged.connect(  
    self._frozen.verticalScrollBar().setValue  
)  
self._frozen.verticalScrollBar().valueChanged.connect(  
    self.tbl_vector.verticalScrollBar().setValue  
)
```

Las dos tablas se colocan lado a lado dentro de un contenedor con *QHBoxLayout*: **_frozen** a la izquierda con ancho fijo, **tbl_vector** a la derecha expandiéndose con el resto del espacio.

Si el usuario arrastra para cambiar el ancho de alguna de las 3 columnas congeladas, el método **_on_frozen_col_resized** recalcula y actualiza el ancho fijo del panel frozen.

10. Filtrado por j e i — sin re-simular

Una vez que la simulación corrió, el usuario puede ajustar dos parámetros de visualización:

- **j (hora desde)**: mostrar solo las filas a partir de ese minuto.
- **i (cantidad de filas)**: cuántas filas mostrar a partir de j.

Cambiar estos valores **no vuelve a simular**. Solo filtra los datos ya calculados. Para hacer el filtrado rápido, al finalizar la simulación se guarda una lista con solo los valores de reloj:

```
self._relojes = [f['reloj'] for f in self.todas_filas]
```

Cuando el usuario cambia j, se usa **bisect_left** para encontrar la posición de la primera fila con reloj $\geq j$ en tiempo $O(\log n)$:

```
start = bisect.bisect_left(self._relojes, j)  
filas = self.todas_filas[start : start + i]
```

bisect_left es búsqueda binaria. Sobre 100.000 filas encuentra la posición correcta en unos 17 pasos en lugar de recorrer toda la lista.

Después de filtrar, se recalculan las columnas dinámicas de empleados a partir de las filas visibles: solo aparecen los empleados que están presentes en ese rango.

11. Tabla de ultima fila y estadísticas

11.1 Tabla de ultima fila

Debajo de la tabla principal hay una tabla de una sola fila que muestra el **estado final del sistema** al terminar la simulación. Usa el mismo *SimModel* cargado con solo la última fila:

```
self.model_ultima.set_content([last], COLS)
```

Altura fija de 96px y sin scroll vertical, para que siempre ocupe el mismo espacio en pantalla sin importar el contenido.

11.2 Tarjetas de estadísticas

El panel de estadísticas muestra 7 métricas globales en tarjetas visuales creadas en Python (no en el .ui). Las métricas son:

Total llegaron · Atendidos · Se fueron (RT) · % que se van · Promedio de espera (min) · Iteraciones · Tiempo simulado (min).

Los valores se calculan a partir de la última fila del vector y de las variables globales de **simulacion.py**:

```
na, nrt, nl, ne, acum = (last.get(k, 0) for k in
    ('n_at', 'n_rt', 'n_lleg', 'n_esp', 'acum_esp'))

stats = {
    'pct_rt':      round(nrt / nl * 100, 2) if nl else 0,
    'prom_espera': round(acum / ne, 3)      if ne else 0,
    ...
}
```

12. Copiar a Excel

El botón *Copiar* vuelca al portapapeles todas las filas visibles en formato **TSV** (valores separados por tabulaciones). Al pegarlo en Excel, cada valor queda en su celda correspondiente.

```
lines = ['\t'.join(str(m.headerData(c, Qt.Horizontal) or '') for c in range(ncol))]
for r in range(nrow):
    lines.append('\t'.join(
        m.data(m.index(r, c), Qt.DisplayRole) or '' for c in range(ncol)))
QApplication.clipboard().setText('\n'.join(lines))
```

Primero agrega la fila de encabezados, luego recorre todas las filas y columnas del modelo leyendo exactamente el mismo texto que se ve en pantalla.

13. Flujo completo de la aplicación

1. Inicio: se carga *main.ui*, se construye el panel frozen, se crean las tarjetas de estadísticas, se conectan las señales de botones y campos.

2. El usuario configura parámetros y presiona *Simular*.

3. on_simular() lee y valida los parámetros. Si hay errores, los muestra en rojo y no continúa. Si

todo está bien, deshabilita el botón, muestra la barra de progreso y lanza *SimWorker* en un hilo aparte.

4. *SimWorker.run()* configura las constantes de *simulacion.py*, llama a *reseteo_estado()* y luego a *simular(tiempo_max)*. Al terminar emite la señal *finished*.

5. *_on_sim_done()* se ejecuta en el hilo principal. Lee el vector de estado, calcula estadísticas, actualiza las tarjetas, rellena la tabla de última fila y llama a *update_table()*.

6. *_on_thread_done()* libera el hilo y el worker con *deleteLater*.

7. *update_table()* aplica los filtros *j* e *i*, calcula columnas dinámicas de empleados, carga los datos en el modelo, actualiza el encabezado y sincroniza el panel frozen.

8. El usuario puede cambiar *j* o *i* para navegar por los datos sin re-simular. Cada cambio dispara *update_table()* de nuevo.

9. El usuario puede copiar las filas visibles al portapapeles con *Copiar*.